



Jadvallarni bog'lash va JOIN. Index, constraint, data integrity. Backup va restore

INNER, LEFT, RIGHT JOIN va ular farqi; bir nechta jadvalni birlashtirish; indekslar va EXPLAIN; cheklovlar va ma'lumotlar yaxlitligi (ON DELETE); pg_dump va pgAdmin orqali zaxira nusxa va tiklash.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Bir nechta jadvaldagi ma'lumotlarni **JOIN** bilan birlashtirishni;
- `INNER`, `LEFT` va `RIGHT JOIN` farqini;
- Qidiruvni tezlashtiruvchi **indekslarni**;
- Ma'lumotlar **yaxlitligi** va cheklovlar qanday himoya qilishini;
- Bazaning **zaxira nusxasini** olish va tiklashni (backup / restore).

Nima uchun JOIN kerak?

26-mavzuda ma'lumotlarni takrorlamaslik uchun ularni turli jadvallarga ajratdik: o'quvchilar alohida, sinflar alohida, baholar alohida. Endi savol: **"Ali Valiyev qaysi sinfda va sinf rahbari kim?"** Javob uchun ikkala jadvalni **birlashtirish** kerak — buni `JOIN` qiladi.



O'xshatish: ikkita ro'yxat

Sizda ikkita qog'oz bor: birinchisida "O'quvchi — sinf raqami", ikkinchisida "Sinf raqami — sinf rahbari". Ali'ning rahbarini bilish uchun birinchi qog'ozdan sinf raqamini topib, ikkinchi qog'ozdan shu raqamni qidirasiz. JOIN aynan shuni avtomatik va millionlab qator uchun bir zumda qiladi.

INNER JOIN

SQL

```
SELECT oquvchilar.ism, oquvchilar.familiya, sinflar.nomi, sinflar.sinf_rahbari
FROM oquvchilar
INNER JOIN sinflar ON oquvchilar.sinf_id = sinflar.id;
```

- `JOIN sinflar` — qaysi jadval bilan birlashtiramiz.
- `ON oquvchilar.sinf_id = sinflar.id` — **qaysi ustunlar bo'yicha** moslashtiramiz (odatda FK = PK).
- `jadval.ustun` — ikki jadvalda bir xil nomli ustun bo'lsa (ikkalasida `id` bor), qaysi biri ekanini aniq ko'rsatish uchun.

Taxalluslar — qisqaroq yozuv

SQL

```
SELECT o.ism, o.familiya, s.nomi AS sinf, s.sinf_rahbari
FROM oquvchilar AS o
JOIN sinflar AS s ON o.sinf_id = s.id          -- JOIN = INNER JOIN
ORDER BY s.nomi, o.familiya;
```

Natija

ism	familiya	sinf	sinf_rahbari
Dilnoza	Aminova	8-A	Karimova Nodira
Zarina	Karimova	8-A	Karimova Nodira
Ali	Valiyev	8-A	Karimova Nodira
Madina	Ergasheva	8-B	Rasulov Akmal
Bobur	Toshmatov	8-B	Rasulov Akmal
...			

JOIN turlari

Agar biror o'quvchining `sinf_id` si bo'sh (NULL) bo'lsa yoki biror sinfda hali o'quvchi bo'lmasa-chi? Shu yerda JOIN turlari farq qiladi:

Tur	Natijada nima bo'ladi	Doira diagrammasi
<code>INNER JOIN</code>	Faqat ikkala jadvalda ham mos keladigan qatorlar	Ikki doiraning kesishmasi
<code>LEFT JOIN</code>	Chap jadvalning HAMMA qatorlari + o'ngdan mos kelganlar (mos kelmasa — NULL)	Butun chap doira
<code>RIGHT JOIN</code>	O'ng jadvalning HAMMA qatorlari + chapdan mos kelganlar	Butun o'ng doira
<code>FULL JOIN</code>	Ikkala jadvalning hamma qatorlari	Ikkala doira

SQL

```
-- Tayyorgarlik: sinfsiz o'quvchi va o'quvchisiz sinf qo'shamiz
INSERT INTO oquvchilar (ism, familiya) VALUES ('Yangi', 'Keldi'); -- sinf_id = NULL
INSERT INTO sinflar (nomi, sinf_rahbari) VALUES ('8-G', 'Azimov B.');
```

-- hali o'quvchisi yo'q

```
-- INNER: "Yangi Keldi" ham, "8-G" ham YO'Q
SELECT o.ism, s.nomi FROM oquvchilar o JOIN sinflar s ON o.sinf_id = s.id;
```

```
-- LEFT: barcha o'quvchilar; "Yangi Keldi" ning sinfi — NULL
SELECT o.ism, s.nomi FROM oquvchilar o LEFT JOIN sinflar s ON o.sinf_id = s.id;
```

```
-- RIGHT: barcha sinflar; "8-G" uchun o'quvchi — NULL
SELECT o.ism, s.nomi FROM oquvchilar o RIGHT JOIN sinflar s ON o.sinf_id = s.id;
```

LEFT JOIN'ning foydali qo'llanilishi: "yo'q"larni topish

SQL

```
-- Hali birorta ham baho olmagan o'quvchilar
SELECT o.ism, o.familiya
FROM oquvchilar o
LEFT JOIN baholar b ON b.oquvchi_id = o.id
WHERE b.id IS NULL;
```



Qaysi JOIN kerakligini bilish uchun so'rang: "Mos kelmagan qatorlar natijada **ko'rinishi kerakmi?**" Yo'q → `INNER`. Chap jadvalning hammasi kerak → `LEFT`. Amalda `RIGHT JOIN` kam ishlatiladi — jadvallar o'rnini almashtirib, `LEFT` bilan yozish mumkin.

Uch jadvalni birlashtirish

SQL

```
-- Kim, qaysi sinfdan, qaysi fandan, qanday baho olgan
SELECT o.ism || ' ' || o.familiya AS oquvchi,
       s.nomi AS sinf,
       b.fan,
       b.baho,
       b.sana
FROM baholar b
JOIN oquvchilar o ON b.oquvchi_id = o.id
JOIN sinflar s ON o.sinf_id = s.id
WHERE b.fan = 'Matematika'
ORDER BY b.baho DESC, oquvchi;
```

Ko'pga-ko'p: to'garaklar

SQL

```
-- Har bir o'quvchi qaysi to'garaklarda (oraliq jadval orqali)
SELECT o.ism, t.nomi AS togarak
FROM oquvchilar o
JOIN azoliklar a ON a.oquvchi_id = o.id
JOIN togaraklar t ON t.id = a.togarak_id
ORDER BY o.ism;
```

Indekslar — qidiruvni tezlashtirish

Jadvalda 1 000 000 qator bo'lsa, `WHERE familiya = 'Karimova'` uchun baza har bir qatorni birma-bir tekshiradi. **Indeks** — maxsus "ko'rsatkich" bo'lib, kerakli qatorlarni darhol topishga yordam beradi.



O'xshatish: kitob oxiridagi ko'rsatkich

"Fotosintez" so'zini darslikda topish uchun 300 betni varaqlaysizmi? Yo'q — kitob oxiridagi alifboli **ko'rsatkich**ga qaraysiz: "Fotosintez — 47, 112-betlar". Indeks — baza uchun xuddi shunday ko'rsatkich.

SQL

```
-- Familiya bo'yicha tez-tez qidiramiz — indeks yaratamiz
CREATE INDEX idx_oquvchilar_familiya ON oquvchilar (familiya);

-- Tashqi kalitlar JOIN'da ishlatiladi — ularga indeks foydali
CREATE INDEX idx_baholar_oquvchi ON baholar (oquvchi_id);

-- Baza so'rovni qanday bajarishini ko'rish
EXPLAIN SELECT * FROM oquvchilar WHERE familiya = 'Karimova';

DROP INDEX idx_oquvchilar_familiya; -- indeksni o'chirish
```

- `PRIMARY KEY` va `UNIQUE` ustunlarga indeks **avtomatik** yaratiladi.
- Indeks o'qishni tezlashtiradi, lekin **yozishni biroz sekinlashtiradi** va joy egallaydi. Shuning uchun faqat **tez-tez qidiriladigan** ustunlarga qo'yiladi.

- Kichik jadvallarda (yuzlab qator) farq sezilmaydi; minglab va millionlab qatorda — juda katta.

Constraint va ma'lumotlar yaxlitligi (data integrity)

Ma'lumotlar yaxlitligi — bazadagi ma'lumotlar har doim **to'g'ri, izchil va ishonchli** bo'lishi. Buni dastur kodiga emas, **bazaning o'ziga** ishonib topshirish kerak — chunki dasturda xato bo'lishi mumkin, baza esa qoidani doim tekshiradi.

Yaxlitlik turi	Nimani kafolatlaydi	Vosita
Obyekt yaxlitligi	Har bir qator noyob va aniqlanadigan	PRIMARY KEY
Havola yaxlitligi	FK doim mavjud qatorga ishora qiladi	FOREIGN KEY (REFERENCES)
Qiymat yaxlitligi	Qiymatlar to'g'ri turda va chegarada	Turlar, NOT NULL , UNIQUE , CHECK

FK qatori o'chirilganda nima bo'ladi?

Variant	Xatti-harakat	Misol
ON DELETE RESTRICT (standart)	O'chirishga ruxsat bermaydi	O'quvchilari bor sinfni o'chirib bo'lmaydi
ON DELETE CASCADE	Bog'liq qatorlar ham o'chadi	O'quvchi o'chsa — baholari ham
ON DELETE SET NULL	FK ustuni NULL bo'ladi	Sinf yopilsa — o'quvchilar "sinfsiz" qoladi

SQL

```
DELETE FROM sinflar WHERE id = 1;
-- ✗ update or delete on table "sinflar" violates foreign key constraint ...
--   Key (id)=(1) is still referenced from table "oquvchilar".
-- Baza "yetim" o'quvchilar paydo bo'lishiga yo'l qo'ymadi ✓
```

Mavjud jadvalga cheklov qo'shish

SQL

```
ALTER TABLE oquvchilar ADD CONSTRAINT email_formati CHECK (email LIKE '%@%.%');  
ALTER TABLE sinflar ADD CONSTRAINT xona_musbat CHECK (xona > 0);
```

Backup va Restore — zaxira nusxa

Kompyuter buzilishi, tasodifiy `DELETE` yoki viruslar — ma'lumot yo'qolishining sabablari ko'p. **Backup** — bazaning nusxasini saqlab qo'yish, **restore** — nusxadan qayta tiklash. Professional qoida: **"Tekshirilmagan zaxira — zaxira emas"** — tiklashni ham sinab ko'ring.

pgAdmin orqali

- 1 **Backup:** maktab bazasi ustida o'ng tugma → **Backup...** → fayl nomi (maktab_2026-10-06.backup) → Format: **Custom** → **Backup**.
- 2 **Restore:** yangi bo'sh baza yarating (maktab_tiklangan) → o'ng tugma → **Restore...** → faylni tanlang → **Restore**.
- 3 Tiklangan bazadagi jadvallar va ma'lumotlarni tekshiring.

Buyruqlar qatori orqali

Terminal

```
# Butun bazani SQL fayl ko'rinishida saqlash  
pg_dump -U postgres -d maktab -f maktab_backup.sql  
  
# Siqilgan (custom) formatda  
pg_dump -U postgres -d maktab -F c -f maktab.backup  
  
# Tiklash  
createdb -U postgres maktab_yangi  
psql -U postgres -d maktab_yangi -f maktab_backup.sql # .sql fayldan  
pg_restore -U postgres -d maktab_yangi maktab.backup # .backup  
fayldan
```

`maktab_backup.sql` faylini oddiy matn muharririda oching — u bazani qaytadan yaratuvchi `CREATE TABLE` va `INSERT` buyruqlaridan iborat!

Amaliy mashg'ulot: mahsulotlar va buyurtmalar bazasi

(19-loyiha)

dokon.sql

```
CREATE TABLE mijozlar (  
    id        SERIAL PRIMARY KEY,  
    ism       VARCHAR(100) NOT NULL,  
    telefon  VARCHAR(20) UNIQUE  
);  
  
CREATE TABLE mahsulotlar (  
    id        SERIAL PRIMARY KEY,  
    nomi     VARCHAR(100) NOT NULL,  
    narx     NUMERIC(12, 2) NOT NULL CHECK (narx > 0),  
    soni     INTEGER NOT NULL DEFAULT 0 CHECK (soni ≥ 0)  
);  
  
CREATE TABLE buyurtmalar (  
    id        SERIAL PRIMARY KEY,  
    mijoz_id  INTEGER NOT NULL REFERENCES mijozlar(id),  
    sana      TIMESTAMP DEFAULT NOW(),  
    holat     VARCHAR(20) DEFAULT 'yangi' CHECK (holat IN ('yangi',  
'yetkazilmoqda', 'yetkazildi', 'bekor'))  
);  
  
CREATE TABLE buyurtma_tarkibi (  
    buyurtma_id  INTEGER REFERENCES buyurtmalar(id) ON DELETE CASCADE, -- ko'pga-ko'p  
    mahsulot_id  INTEGER REFERENCES mahsulotlar(id),  
    soni         INTEGER NOT NULL CHECK (soni > 0),  
    narx         NUMERIC(12, 2) NOT NULL, -- sotilgan  
    paytdagi_narx NUMERIC(12, 2) NOT NULL,  
    PRIMARY KEY (buyurtma_id, mahsulot_id)  
);  
  
INSERT INTO mijozlar (ism, telefon) VALUES  
    ('Aziz', '+998901111111'), ('Nilufar', '+998902222222'), ('Kamol',  
'+998903333333');  
  
INSERT INTO mahsulotlar (nomi, narx, soni) VALUES  
    ('Daftar', 5000, 200), ('Ruchka', 3000, 500), ('Ryukzak', 180000, 15),  
    ('Kalkulyator', 95000, 0);  
  
INSERT INTO buyurtmalar (mijoz_id, holat) VALUES (1, 'yetkazildi'), (2,  
'yangi'), (1, 'yangi');  
  
INSERT INTO buyurtma_tarkibi VALUES  
    (1, 1, 10, 5000), (1, 2, 5, 3000), (2, 3, 1, 180000), (3, 1, 2, 5000);
```

Quyidagi so'rovlarni yozing:

- 1 Har bir buyurtma: raqami, mijoz ismi, sanasi va holati.
- 2 1-buyurtma tarkibi: mahsulot nomi, soni, narxi va `soni * narx` (summa).
- 3 Hali birorta ham buyurtma bermagan mijozlar (LEFT JOIN + IS NULL).
- 4 Hech qachon sotilmagan mahsulotlar.
- 5 Bazaning zaxira nusxasini oling, `mahsulotlar` jadvalidan bitta qatorni o'chiring, so'ng nusxadan yangi bazaga tiklab, qator o'z joyida ekanini tekshiring.

▼ Javoblar (1–4)

SQL

```
SELECT b.id, m.ism, b.sana, b.holat
FROM buyurtmalar b JOIN mijozlar m ON m.id = b.mijoz_id;

SELECT p.nomi, t.soni, t.narx, t.soni * t.narx AS summa
FROM buyurtma_tarkibi t JOIN mahsulotlar p ON p.id = t.mahsulot_id
WHERE t.buyurtma_id = 1;

SELECT m.ism FROM mijozlar m
LEFT JOIN buyurtmalar b ON b.mijoz_id = m.id
WHERE b.id IS NULL;

SELECT p.nomi FROM mahsulotlar p
LEFT JOIN buyurtma_tarkibi t ON t.mahsulot_id = p.id
WHERE t.mahsulot_id IS NULL;
```

Ko'p uchraydigan xatolar

- ✗ column reference "id" is ambiguous — ikki jadvalda ham `id` bor; `o.id` yoki `s.id` deb aniq yozing.
- ✗ `ON` shartini unutish yoki noto'g'ri ustunlarni bog'lash (`o.id = s.id`) — natija mantiqsiz bo'ladi.
- ✗ LEFT JOIN'dan keyin o'ng jadval bo'yicha `WHERE` qo'yib, uni INNER'ga aylantirib qo'yish.

- ❌ Zaxira nusxani o'sha kompyuterning o'zida saqlash — kompyuter buzilsa, nusxa ham yo'qoladi. Boshqa joyga (fleshka, bulut) ham ko'chiring.

O'zingizni tekshiring

? Faqat ikkala jadvalda ham mos keladigan qatorlarni qaytaruvchi JOIN?

- A. INNER JOIN
- B. LEFT JOIN
- C. RIGHT JOIN
- D. FULL JOIN

? Barcha o'quvchilarni (sinfi bo'lmaganlarini ham) sinf nomi bilan chiqarish uchun?

- A. oquvchilar INNER JOIN sinflar
- B. sinflar INNER JOIN oquvchilar
- C. oquvchilar LEFT JOIN sinflar
- D. oquvchilar RIGHT JOIN sinflar

? Indeksning asosiy vazifasi?

- A. Ma'lumotlarni shifrlash
- B. Qidiruvni tezlashtirish
- C. Zaxira nusxa olish
- D. Jadvallarni bog'lash

? O'quvchi o'chirilganda uning baholari ham avtomatik o'chishi uchun FK'ga nima qo'shiladi?

- A. `ON DELETE RESTRICT`
- B. `ON DELETE SET NULL`
- C. `NOT NULL`
- D. `ON DELETE CASCADE`

? PostgreSQL bazasining zaxira nusxasini oluvchi buyruq?

- A. `pg_backup`
- B. `pg_dump`
- C. `COPY DATABASE`
- D. `SAVE DATABASE`



Uyga vazifa

- Kutubxona bazangizda (27-mavzu uyga vazifasi): har bir ijara — kitob nomi, muallif, o'quvchi ismi; hali qaytarilmagan kitoblar; hech kim olmagan kitoblar.
- Haftada bir marta avtomatik backup qilish nima uchun muhimligini va qanday tashkil qilish mumkinligini yozing.



Yodda tuting

- `SELECT ... FROM a JOIN b ON a.fk = b.id`; taxalluslar: `FROM oquvchilar o`.
- INNER — faqat moslar; LEFT — chapning hammasi; RIGHT — o'ngning hammasi. "Yo'q"larni topish: `LEFT JOIN + IS NULL`.
- Indeks — tez qidiruv uchun ko'rsatkich (`CREATE INDEX`); PK/UNIQUE'da avtomatik.
- Yaxlitlik: PK, FK, NOT NULL, UNIQUE, CHECK; `ON DELETE CASCADE / RESTRICT / SET NULL`.
- Backup: `pg_dump` / pgAdmin Backup; restore: `psql -f`, `pg_restore`. Tiklashni sinab ko'ring!

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. Faqat ikkala jadvalda ham mos keladigan qatorlarni qaytaruvchi JOIN? — **A) INNER JOIN**
2. Barcha o'quvchilarni (sinfi bo'lmaganlarini ham) sinf nomi bilan chiqarish uchun? — **C) oquvchilar LEFT JOIN sinflar**
3. Indeksning asosiy vazifasi? — **B) Qidiruvni tezlashtirish**
4. O'quvchi o'chirilganda uning baholari ham avtomatik o'chishi uchun FK'ga nima qo'shiladi? — **D) ON DELETE CASCADE**
5. PostgreSQL bazasining zaxira nusxasini oluvchi buyruq? — **B) pg_dump**